

מועד א' 2025

שאלה 1

א. נעבור על המערך איבר-איבר ונחפש את המספר החיובי הקטן ביותר במערך. נשמור אותו במשתנה בשם positiveMin. ריצה על כל האיברים ומציאת האיבר הזה לוקחת $O(n)$ כי היא מאוד דומה לאלגוריתם מציאת מינימום כללי של מערך בגודל n . לאחר מכן נריץ את Partition על האיבר הזה. בסיום ה-Partition, כל המספרים שקטנים ממש ממנו (שהם כולם שליליים כי מצאנו את החיובי המינימלי) נמצאים משמאל, וכל המספרים החיוביים נמצאים מימין. סיבוכיות הריצה היא $O(n)$ למציאת המינימום ו- $O(n)$ לביצוע Partition כך שמקבלים סה"כ $O(n)$. תוספת הזיכרון היא $O(1)$ כי בסך הכל הוספנו משתנה סקלרי.

ב.

$$T(n) = 27T\left(\frac{n}{3}\right) + 81n^3 + 243n^2 \log^6(n)$$

זוהי משוואה שניתן לפתור בעזרת שיטת האב בה:

$$a = 27$$

$$b = 3$$

$$f(n) = 81n^3 + 243n^2 \log^6(n) = \Theta(n^3)$$

במקרה שלנו:

$$n^{\log_b(a)} = n^{\log_3(27)} = n^3$$

זהו מקרה 2 של שיטת האב, לכן:

$$T(n) = \Theta(n^{\log_b(a)} \log(n)) = \Theta(n^3 \log(n))$$

שאלה 2

א. נמייך רק את המערך הקטן בעזרת MergeSort, ונשאיר את הגדול כפי שהוא. נניח בלי הגבלת הכלליות ש- $m < n$ ואנחנו ממיינים את המערך בגודל m . סיבוכיות המיין היא:

$$O(m \log(m)) = O(m \log(\min\{m, n\})) < O((m+n) \log(\min\{m, n\}))$$

כאשר השוויון הראשון נובע מהעובדה ש- $\min\{m, n\} = m$. כעת יש לנו מערך ראשון ממין ומערך שני לא ממין. נניח שהמערך הממין הוא A (שוב, בלי הגבלת הכלליות). נסתכל על האיבר הראשון ב- B (המערך הלא ממין) ונחפש איבר ב- A המקיים:

$$A[i] + B[j] = C$$

בעזרת חיפוש בינארי. כלומר, לכל איבר שנבדוק ב- A , נראה אם מתקיים $A[i] + B[j] = C$. אם $A[i] + B[j] < C$ אז נקפיץ את i ימינה כמו בחיפוש בינארי. אם $A[i] + B[j] > C$ אז נקפיץ את i שמאלה כמו בחיפוש בינארי. אחרי שסיימנו את החיפוש הבינארי הזה, נבצע חיפוש בינארי שוב (כאשר j עדיין מסתכל על אותו איבר ב- B), רק הפעם אנחנו בוחנים את $A[i] - B[j]$. אם $A[i] - B[j] < C$ אז נקפיץ את i ימינה כמו בחיפוש בינארי. אם $A[i] - B[j] > C$ אז נקפיץ את i שמאלה כמו בחיפוש בינארי. לאחר שני חיפושים בינאריים לא מוצלחים, נגדיל את j באחת ונבצע שוב שני חיפושים בינאריים נפרדים באותו אופן. במקרה הגרוע (בו לא מצאנו אינדקסים i ו- j עם התנאים הדרושים) יתבצעו n פעמים שני חיפושים בינאריים נפרדים על מערך בגודל m . סיבוכיות הקוד תהיה מורכבת מ-MergeSort בגודל m ו- $2n$ חיפושים בינאריים בגודל m . הסיבוכיות הכוללת היא:

$$O(m \log(m)) + O(2n \log(m)) = O((n+m) \log(m))$$

וכמובן אם n היה המערך הקטן יותר היינו מקבלים:

$$O(n \log(n)) + O(2m \log(n)) = O((n+m) \log(n))$$

כך שהסיבוכיות במקרה הכללי היא:

$$O((n+m) \log(\min\{m, n\}))$$

ב. נשלח את כל האיברים של המערך B לתוך טבלת גיבוב. מילוי טבלת הגיבוב לוקח $O(n)$. לאחר מכן נעבור על כל האיברים ב- A . עבור כל אחד מהם נבדוק אם המספר $A[j] - C$ או $C - A[j]$ קיים בטבלת הגיבוב. בממוצע, חיפוש אם איבר מסוים קיים בטבלת גיבוב לוקח $O(1)$ כך שמעבר על m איברים יקח $O(m)$. סך הסיבוכיות בקוד (בניית הטבלה ומעבר על A) לוקח $O(n+m)$.

שאלה 3

נשתמש בעץ אדום-שחור. כמעט בלי שינויים.

נשמור מצביע לצומת הקטן ביותר בעץ. ניתן לתחזק אותו בכל הכנסה ומחיקה בכך שנשתמש בפקודת FindMin של עץ בינארי מיד לאחר ההכנסה/מחיקה.

בנוסף, נשמור מצביע לצומת השני הקטן ביותר על פי החוקיות הבאה:

אם לצומת עם המצביע הראשון יש תת עץ ימני, נחפש בתת העץ הימני את המינימום בעזרת FindMin. בהכרח צומת זה יהיה המינימום השני.

אם לצומת עם המצביע הראשון אין תת עץ ימני, אז אבא שלו הוא המינימום השני.

Insert - פקודת RB-Insert כפי שראינו בכיתה - פועלת ב- $O(\log(n))$.

Delete - פקודת RB-Delete כפי שראינו בכיתה - פועלת ב- $O(\log(n))$.

Find - פקודת TreeSearch כפי שראינו בכיתה - פועלת ב- $O(\log(n))$.

Min1_2 - נחזיר את הערך של שני המצביעים שלנו - זה $O(1)$.

Print_Max_k - נבצע סריקה תוכית הפוכה של עץ חיפוש בינארי. כלומר סריקה של בן ימני, ואז שורש, ואז בן שמאלי.

בהכרח הצמתים שנעבור עליהם יהיו ממוינים הפוך (כלומר מהגדול לקטן).

נפסיק את הסריקה לאחר k צמתים ונקבל הדפסה של k הצמתים הכי גדולים לפי סדר מהגדול לקטן.

סיבוכיות הפקודה תהיה $O(k)$ כי יש לנו k צמתים לעבור עליהם ולהדפיס. גם ראינו בכיתה שסריקה תוכית (אמנם הפוכה) של n צמתים לוקחת $O(n)$.

שאלה 4

א. נחפש את המספר שלנו בחיפוש בינארי.

נתחיל באינדקס $i = \frac{n}{2}$ ונבדוק אם $A[i] = i$.

אם $A[i] < i$, אז בהכרח נקודת שבת צריכה להיות בחצי הימני (אם קיימת כמובן). נזרוק את החצי השמאלי של המערך ונמשיך לחפש באמצע החצי הימני של המערך (כמו בחיפוש בינארי רגיל).

הסיבה שנקודת שבת חייבת להיות בחצי הימני היא שהמערך מכיל מספרים **שונים** זה מזה. אם למשל מצאנו את הערך 2 באינדקס 3, לא יכול להיות שקיימת נקודת שבת מצד שמאל כי עבור כל צעד שמאלה שנבצע (וכך נוריד את האינדקס ב-1), הערך במערך ירד ב-1 לפחות.

אם $A[i] > i$, אז בהכרח נקודת שבת צריכה להיות בחצי השמאלי. הפעם נזרוק את החצי הימני. אם לא מצאנו נקודת שבת, נחזיר 0.

ב. סיבוכיות הקוד היא כסיבוכיות חיפוש בינארי רגיל - $O(\log(n))$.